KnowFeat: Knowledge-Guided Feature Engineering with Auditable Provenance via LLM Agents

Chengsong You[1], Wangyue Li[1], Weiqiao Que[1], Qizhou Chen[1], Wei Deng[2], Feng Zhu[3], Xiaofeng He[1,*]
[1]East China Normal University, Shanghai, China
{51275901122, 52285901017}@stu.ecnu.edu.cn, weiqiaoque@gmail.com, 582476065@qq.com, hexf@cs.ecnu.edu.cn
[2]Southwestern University of Finance and Economics, Chengdu, China
dengwei@swufe.edu.cn
[3]Digital Currency Department, Postal Savings Bank of China, Beijing, China
Jason_bank@126.com

***Abstract*—Automated feature engineering with large language models (LLMs) can produce semantically meaningful features for tabular data, yet existing methods lack structured domain knowledge, rigorous verification, and auditable provenance. We propose KnowFeat, a knowledge-guided feature engineering framework that addresses all three gaps. KnowFeat organizes domain knowledge into a five-type hierarchy (schema metadata, regulatory indicators, detection rules, expert opinions, and court document evidence) and injects it as structured context into an LLM agent. A three-stage verification pipeline filters candidates through code execution, statistical quality checks, and model effectiveness evaluation. Every accepted feature carries a provenance card linking its design to specific knowledge assets. On seven public benchmarks, KnowFeat achieves the best average rank (1.4) among eight methods. On a realistic anti-money laundering dataset (synthesized to match the statistical properties of production digital currency transactions) with 200K transactions, 18 verified features improve Recall by +2.1pp and F1 by +1.5pp, outperforming methods that generate over 1,000 unverified features. Code and data are available at https://anonymous.4open.science/r/knowfeat-C967/ and https://huggingface.co/datasets/anonymous-7219/knowfeat.**

I. Introduction

Feature engineering is among the most impactful steps in building machine learning models for tabular data [1]. Well-designed features improve model accuracy, reduce overfitting, and provide interpretable signals for downstream tasks. However, manual feature engineering requires deep domain expertise and does not scale across diverse applications.

Traditional automated feature engineering (AutoFE) methods such as OpenFE [2] and AutoFeat [3] apply systematic transformations (mathematical operations, aggregations, and feature crosses) to existing columns, then select the most predictive ones. These methods are purely data-driven: they explore a predefined transformation space without understanding what the features *mean* or why certain transformations might be useful. In practice, understanding the *semantic* meaning of features is crucial in regulated domains such as finance and healthcare, where stakeholders must know *what* a feature represents before deploying it.

The emergence of large language models (LLMs) has opened a new paradigm for feature engineering. Methods such as CAAFE [4], FeatLLM [5], and LLM-FE [6] use LLMs to generate feature transformation code from dataset descriptions, leveraging the LLM's broad world knowledge to propose semantically meaningful features. However, current LLM-based methods share three limitations:

(1) Lack of hierarchical, structured domain knowledge. Most methods rely on the LLM's parametric knowledge, which may be outdated or incomplete for specialized domains. None organizes external domain knowledge (regulatory indicators, detection rules, expert opinions, and case evidence) into a hierarchical format for systematic injection.

(2) Insufficient verification. Existing verification mechanisms remain incomplete. CAAFE uses a logistic regression proxy; LLM-FE uses downstream accuracy without statistical filtering; FeatLLM performs no model-based validation. No method combines code execution, statistical quality filtering, and full model evaluation in a unified pipeline.

(3) Limited auditability. In regulated industries, model features must be auditable. Decision-makers need to understand *why* a feature was created and *what evidence* supports it. Existing methods lack structured provenance that traces each feature to specific knowledge assets with detailed reasoning chains.

We propose **KnowFeat** (**Know**ledge-guided **Feat**ure engineering), a framework that addresses all three limitations:

- **Hierarchical Knowledge Injection.** We organize domain knowledge into five types of increasing specificity (schema metadata, risk indicators, detection rules, expert opinions, and court document evidence) and inject them as structured context into the LLM agent.
- **Three-Stage Verification Pipeline.** Each candidate passes through L1 (code execution sandbox), L2 (statistical quality with noise-calibrated thresholds), and L3 (model effectiveness via XGBoost across 5 seeds).
- **Auditable Provenance.** Every accepted feature carries a structured provenance card recording the knowledge assets, reasoning chain, computation steps, and verification results.

We evaluate KnowFeat on seven public tabular datasets from OpenML and a realistic anti-money laundering (AML) dataset, synthesized to match production digital currency transaction

*Corresponding author.

patterns, containing 200,000 transactions with 93 baseline features. Key findings:

- On public datasets, KnowFeat achieves the best average rank (1.4) among eight methods with 12–46 features per dataset, compared to 98–1,124 for FeatLLM.
- On the AML dataset, 18 knowledge-guided features improve Recall by +2.1pp and F1 by +1.5pp over the 93-feature baseline, outperforming all baselines.
- Ablation experiments confirm that domain knowledge is essential: the schema-only variant generates zero accepted features on one dataset, and the full hierarchy outperforms schema-only on 5 of 6 remaining datasets.

The remainder of this paper is organized as follows. Section II reviews related work. Section III presents the KnowFeat framework. Section IV describes our experimental setup and results. Section V illustrates knowledge-grounded features through case studies. Section VI discusses practical considerations for deployment. Section VII concludes with limitations and future directions.

II. Related Work

A. Traditional Automated Feature Engineering

Traditional AutoFE methods search a *syntactic* transformation space. Deep Feature Synthesis [7] applies aggregation primitives along relational paths; ExploreKit [8] enumerates operators and ranks candidates via meta-learning; AutoFeat [3] uses polynomial expansion with L1 selection; and OpenFE [2] frames feature generation as an operator-operand search with LightGBM-based importance ranking. End-to-end AutoML systems such as Auto-sklearn [9] and FLAML [10] include feature preprocessing but do not generate semantically new features. These methods apply predefined mathematical operators without *semantic* understanding: `age × income` is treated identically to `age × zip_code`, even though only the former has financial meaning. They cannot generate domain-specific features (e.g., "transaction velocity" in anomaly detection) that require conceptual understanding [11].

B. LLM-Based Feature Engineering

Large language models [12], [13] have enabled a shift from syntactic to semantic feature generation. CAAFE [4] iteratively prompts an LLM with dataset descriptions and evaluation feedback to generate Python code for new features, using logistic regression as a proxy evaluator. FeatLLM [5] takes a two-pass approach: first extracting natural language rules from the LLM, then converting these rules into binary indicator features. LLM-FE [6] frames feature engineering as an evolutionary optimization problem, using LLMs to iteratively generate and refine features through structured prompts with model performance feedback. OCTree [14] guides LLM-based feature generation with decision tree reasoning, using tree structure feedback to steer the generation process. In the financial domain, RiskTagger [15] uses an LLM agent with regulatory knowledge to annotate Web3 crypto laundering behaviors, demonstrating that domain knowledge can enhance LLM-based data annotation.

TABLE I
Comparison of automated feature engineering methods. ✓: fully supported; ○: partially supported; blank: not supported; –: not applicable.

Property	Open FE	Auto Feat	CAA FE	Feat LLM	LLM-FE	OCTree	Know Feat
Semantic underst.			✓	✓	✓	✓	✓
Hier. knowledge					○		✓
Exec. verific.	–	–					✓
Statistical filt.	✓	✓					✓
Model validation			○		✓	✓	✓
Provenance track.							✓
Adaptive search					✓	✓	✓

○ Hier. knowl.: LLM-FE=dataset desc. only. Model valid.: CAAFE=LogReg proxy; OCTree=DT feedback.

These methods vary in their use of domain knowledge and verification. LLM-FE incorporates limited domain context (dataset descriptions), but none organizes knowledge into a hierarchical, structured format for systematic injection. Verification also varies: CAAFE uses a weak proxy model, LLM-FE uses downstream model accuracy, OCTree uses decision tree feedback, and FeatLLM performs no model-based validation. None combines code execution, statistical quality filtering, and model evaluation into a unified pipeline, nor offers structured provenance that traces each feature to specific knowledge assets.

Tool-augmented LLM agents [16] have been applied to code generation [17], software engineering [18], data analysis [19], and scientific discovery [20]. In data science, AIDE [21] and DS-Agent [22] automate end-to-end pipelines but do not address knowledge-grounded feature generation. KnowFeat builds on the tool-use paradigm by equipping an LLM agent with specialized tools for data exploration, feature generation with provenance, and multi-stage verification.

Table I summarizes the key differences among related methods. KnowFeat is the first to combine hierarchical structured domain knowledge, multi-stage verification (L1–L3), and full feature provenance tracking within an LLM-based feature engineering framework.

C. Domain Knowledge Integration in Machine Learning

Knowledge-infused learning [23] incorporates external knowledge graphs into neural architectures, and transfer learning [24] transfers knowledge through pretrained representations, but both target unstructured data rather than tabular features. In financial AML, domain experts manually encode risk indicators, structuring patterns, and network centrality measures as features [25]. Regulatory bodies publish red-flag typologies (e.g., FATF guidelines [26]), and institutions maintain internal detection rules from compliance experience and legal proceedings [27]. This rich, structured knowledge has not been leveraged by automated feature engineering methods.

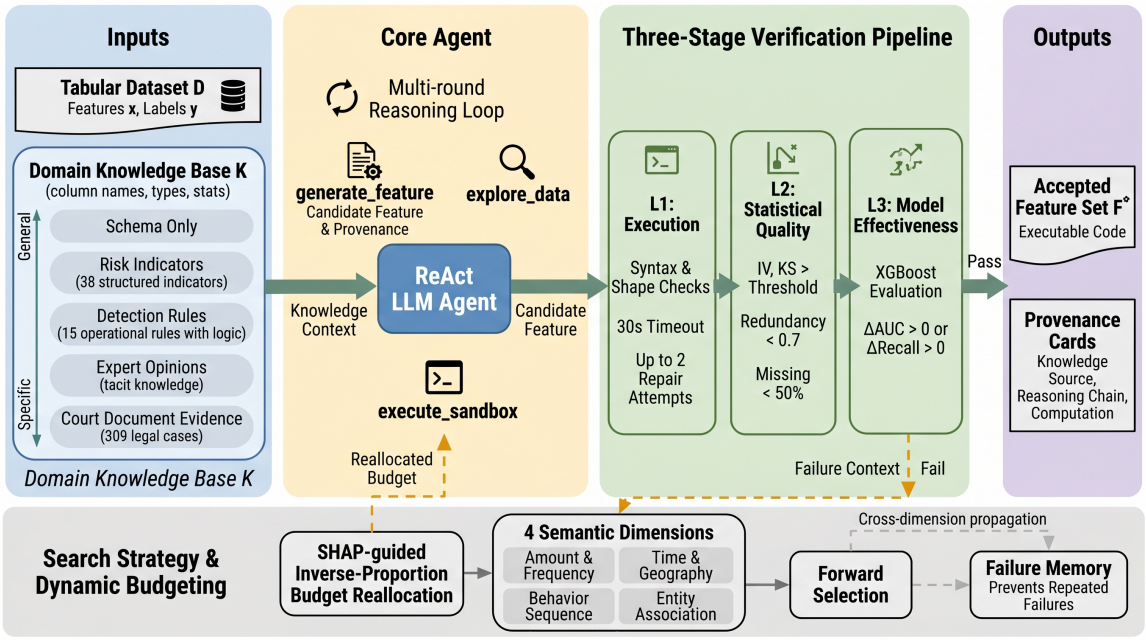


Fig. 1. Overview of the KnowFeat framework. The LLM agent generates features guided by hierarchical domain knowledge. Each candidate is verified through a three-stage pipeline (L1–L3). SHAP feedback dynamically reallocates the exploration budget across semantic dimensions.

III. Methodology

This section presents the KnowFeat framework in detail. We first provide an overview of the pipeline (Section III-A), then describe each component: hierarchical knowledge injection (Section III-B), the LLM agent with specialized tools (Section III-C), the three-stage verification pipeline (Section III-D), the intelligent search strategy (Section III-E), and auditable provenance output (Section III-F).

A. Framework Overview

KnowFeat is a multi-round, knowledge-guided feature engineering framework built on an LLM agent. Given a tabular dataset with baseline features and target labels, along with a domain knowledge base, KnowFeat generates a set of new features such that appending them to the original data improves predictive performance on a target metric (e.g., AUC or Recall).

The overall pipeline operates as follows (see Figure 1 for an illustration):

1) **Initialization.** Load the dataset, compute baseline model performance, and calibrate statistical thresholds via noise injection (Section III-D).
2) **Knowledge Assembly.** Organize domain knowledge $\mathcal{K}$ into a five-type hierarchy and inject the selected types into the agent's context (Section III-B).
3) **Multi-Round Generation.** For each round $r = 1, \ldots, R_{\max}$:
   a) Allocate per-dimension budgets based on SHAP feedback from the previous round.
   b) For each semantic dimension (Section III-E), the LLM agent explores the data, proposes candidate features grounded in knowledge assets, and submits them for verification.
   c) Each candidate passes through the L1→L2→L3 pipeline. Accepted features are added to the baseline; rejected features enter the failure memory.
   d) Forward selection discovers feature combinations that are individually weak but collectively effective.
4) **Convergence.** The process terminates when no new features are accepted for C consecutive rounds (default $C = 2$) or the maximum round count $R_{\max}$ is reached.
5) **Output.** The accepted feature set $\mathcal{F}^*$, along with executable code and auditable provenance cards for each feature (Section III-F).

B. Hierarchical Knowledge Injection

KnowFeat systematically integrates external domain knowledge into the feature generation process. We organize knowledge into five hierarchical types of increasing specificity, from general metadata to concrete case evidence:

(i) Schema Metadata. The agent receives only the dataset schema: column names, data types, and basic statistics (mean, cardinality, missing rate). No domain knowledge is provided. This serves as the ablation baseline, testing what the LLM can generate from metadata alone.

(ii) Risk Indicators. Structured risk indicators derived from regulatory guidelines and industry standards. Each indicator specifies a risk dimension, risk class, relevant data field, and directionality. For the AML domain, we compile 38 indicators (IND_01 through IND_38) covering transaction amount thresholds, frequency patterns, temporal anomalies,

and network centrality measures. For example, IND_05 flags accounts whose single-day transaction count exceeds a regulatory threshold, while IND_15 identifies abnormal transaction modes such as fan-in and many-to-many patterns.

(iii) Detection Rules. Operational detection rules that encode specific logic patterns. Each rule includes a rule ID, natural language description, logical expression, confidence level, and a Python implementation hint. We compile 15 rules (RULE_001 through RULE_015) from compliance manuals and internal documentation. For example, RULE_006 encodes the "scatter-in, gather-out" layering pattern.

(iv) Expert Opinions. Insights from domain expert interviews that capture tacit knowledge (heuristics, edge cases, and domain intuitions) difficult to formalize as rules but valuable for feature design. We collect 12 expert opinions (EXP_01 through EXP_12). For example, EXP_05 notes that "border region transactions combined with nighttime activity are a hallmark of cross-border laundering."

(v) Court Document Evidence. Risk factors extracted from 309 real court judgments via a key-clue extraction pipeline. This is the most concrete type, grounding feature generation in case evidence (e.g., pyramid-shaped fund network patterns independently validated by legal proceedings).

The five types can be freely combined via a knowledge filter parameter, enabling ablation study of each type's contribution. In the "full" configuration (default), all types are included. Each knowledge item is assigned a unique identifier for provenance tracking.

C. LLM Agent with Specialized Tools

KnowFeat employs an LLM agent (DeepSeek-V4-0324-Flash in our implementation, with temperature=0 for near-deterministic outputs) that interleaves reasoning with tool calls following the ReAct paradigm [16]. The agent is equipped with three specialized tools:

Tool 1: Data Exploration. Executes Python code on the dataset in a sandboxed environment to inspect distributions, compute correlations, and identify patterns before proposing features. The tool returns printed output or return values to the agent.

Tool 2: Feature Registration. Registers a candidate feature with mandatory provenance fields:

- `feature_name`: A descriptive identifier.
- `code`: Executable Python code that computes the feature from raw data.
- `explanation`: A natural language description of what the feature captures.
- `reasoning`: A structured chain linking knowledge assets to the feature design, formatted as "Step N: Because [knowledge ID] indicates [insight], we compute [transformation]..."
- `knowledge_ids`: A list of specific knowledge asset identifiers (e.g., RULE_001, IND_05, EXP_03) that inspired the feature.
- `computation_steps`: Step-by-step description from raw fields to the final feature value.
- `data_fields_used`: The raw columns consumed by the feature.

Tool 3: Verification Trigger. Triggers the L1→L2→L3 verification pipeline on a registered feature and returns the result (accepted/rejected with detailed reasons). The agent must call Feature Registration followed immediately by Verification Trigger for each candidate; batch submission is prohibited to ensure tight feedback loops.

The agent's prompt instructs it to: (1) reason about which knowledge assets are relevant to the current dimension and budget, (2) explore the data to validate assumptions, (3) propose a feature grounded in specific knowledge items, and (4) submit it for verification and react to the feedback.

D. Three-Stage Verification Pipeline

Each candidate feature passes through three progressively stricter verification stages:

1) L1: Code Execution Sandbox: The generated Python code is executed in an isolated sandbox with a 30-second timeout. L1 checks for:

- Syntax and runtime errors.
- Output shape consistency (one value per sample).
- Presence of required identifier columns.
- Data type validity (numeric output).

If L1 fails, the agent receives the error message and is allowed up to 2 automatic repair attempts before the feature is permanently rejected.

2) L2: Statistical Quality Filtering: The feature's statistical properties are evaluated against automatically calibrated thresholds. To establish these thresholds for discriminative power, we generate $N_{\text{noise}} = 100$ random features (drawn from standard distributions), compute their quality metrics, and set the acceptance threshold at the P_{noise}-th percentile (default $P_{\text{noise}} = 95$). This ensures that accepted features are statistically better than random noise. L2 checks four criteria:

- **Information Value (IV)** [25]: Must exceed the noise-calibrated threshold, ensuring the feature has meaningful discriminative power between positive and negative classes.
- **Kolmogorov–Smirnov (KS) Statistic** [11]: Must exceed the noise-calibrated threshold, measuring the maximum distributional separation between classes.
- **Redundancy**: Maximum absolute Pearson correlation with all existing accepted features must be below 0.7, preventing near-duplicate features from inflating the feature set.
- **Missing Rate**: The proportion of NaN values must be at most 50%, ensuring sufficient coverage for model training.

The noise-calibrated thresholds apply to IV and KS; the redundancy and missing rate thresholds are fixed.

3) L3: Model Effectiveness: The feature is evaluated for its actual contribution to predictive performance. We train an XGBoost [28] model on the baseline features augmented with the candidate feature, using stratified 80/20 train-test

splits across 5 random seeds $\{42, 123, 456, 789, 2024\}$. The acceptance criterion depends on the task:

- **Classification (public datasets)**: $\Delta\text{AUC} > 0$, i.e., the augmented model must achieve strictly higher AUC than the baseline.
- **AML detection**: $\Delta\text{Recall} > 0$ *and* Precision $\geq 90\%$, ensuring that the feature improves detection without sacrificing precision.

The three-stage design is intentionally progressive: L1 is cheap (milliseconds), L2 is moderate (seconds), and L3 is expensive (minutes per feature). By filtering early, we avoid wasting computational resources on clearly defective candidates.

E. Intelligent Search Strategy

To systematically explore the feature space, KnowFeat employs a multi-component search strategy.

1) Dimension-Based Budgeting: We decompose the feature space into D semantic dimensions relevant to the target domain (default $D = 4$ for the AML domain):

1) **Amount & Frequency**: Features related to monetary values and transaction counts.
2) **Time & Geography**: Features capturing temporal patterns and spatial distributions.
3) **Behavior Sequence**: Features encoding sequential patterns in user actions.
4) **Entity Association**: Features derived from relationships between entities (accounts, counterparties).

Each dimension receives an initial budget of B_0 candidates per round (default $B_0 = 3$). After each round, SHAP [29] (SHapley Additive exPlanations) analysis is performed on the current model to estimate each dimension's contribution to predictions.

2) SHAP-Guided Budget Reallocation: Let s_j denote the mean absolute SHAP value for dimension j's accepted features, $B_{\min}$ the minimum per-dimension budget (default 1), and B_{total} the total budget for the round. We reallocate the budget for round $r{+}1$ using an *inverse-SHAP weighting* strategy:

$$B_j^{(r+1)} = B_{\min} + (B_{\text{total}} - D \cdot B_{\min}) \cdot \frac{1/s_j}{\sum_{j'=1}^{D} 1/s_{j'}} \quad (1)$$

This ensures that underrepresented dimensions (those contributing least to the current model) receive more exploration budget, promoting balanced feature coverage.

3) Failure Memory: A failure memory $\mathcal{M}_{\text{fail}}$ records all rejected features along with their rejection reasons. Before each generation step, the K most recent failures for the current dimension (default $K = 10$) are injected into the agent's prompt, preventing the regeneration of previously rejected ideas and guiding the agent toward novel directions.

4) Forward Selection: Individual features may fail L3 yet contribute positively when combined with others. We apply greedy forward selection on L2-passed but L3-failed candidates, sorted by L2 quality scores. Each candidate is added to the accepted set if the augmented set improves the target metric, capturing synergistic feature interactions.

5) Cross-Dimension Propagation: After each round, the top-K_{cross} features (default $K_{\text{cross}} = 5$) by SHAP importance are shared across all dimensions as "inspiration prompts," encouraging the agent to adapt successful patterns from one dimension to others.

F. Auditable Provenance Output

Every generated feature is accompanied by a structured *provenance card* that enables full auditability. Each provenance card records:

- **Knowledge assets**: The specific identifiers (e.g., RULE_006, IND_15, EXP_05) that inspired the feature design.
- **Reasoning chain**: A step-by-step explanation of how domain knowledge led to the feature, formatted as "Step N: Because [knowledge ID] indicates [insight], we compute [transformation]..."
- **Computation steps**: The transformation pipeline from raw data fields to the final feature value.
- **Verification results**: L1/L2/L3 outcomes including IV, KS, redundancy, and ΔAUC/ΔRecall.

This traceability is essential for regulated domains: compliance officers can audit *why* each feature exists, *what evidence* supports it, and *how* it was verified. Section V illustrates provenance cards through concrete examples.

IV. Experiments

We evaluate KnowFeat on both public tabular benchmarks and a realistic anti-money laundering (AML) dataset. Our experiments aim to answer four research questions:

- **RQ1**: Does KnowFeat improve predictive performance on public datasets?
- **RQ2**: How does KnowFeat perform on a realistic AML task with severe class imbalance?
- **RQ3**: What is the contribution of domain knowledge?
- **RQ4**: How does KnowFeat balance feature quantity vs. quality?

A. Experimental Setup

1) Datasets: We use eight datasets summarized in Table II: seven public benchmarks from OpenML [30] and one realistic AML dataset.

The seven public datasets are standard tabular classification benchmarks obtained via the OpenML Python API [30]. SimECNY is a synthetic AML benchmark calibrated against real digital currency (e-CNY) transaction patterns from the Postal Savings Bank of China. Its 93 baseline features, transaction topology, and class distribution (2.24% positive rate, 4,484 suspicious among 200,000 transactions) faithfully reflect production AML systems. Due to financial privacy regulations, high-fidelity simulation is standard practice in AML research [27]. Importantly, the domain knowledge base (38 indicators, 15 rules, 12 expert opinions, 309 court documents) is derived entirely from real regulatory guidelines,

TABLE II
DATASET STATISTICS.

Dataset	Samples	Feat.	Pos%	Domain
credit_g	1,000	20	70.0	Finance
diabetes	768	8	34.9	Healthcare
blood_transfusion	748	4	23.8	Healthcare
heart	918	11	55.3	Healthcare
bank_marketing	45,211	16	11.7	Marketing
breast_w	683	9	35.0	Healthcare
adult	48,842	14	23.9	Census
SimECNY (AML)	200,000	93	2.24	Finance

compliance manuals, expert interviews, and publicly available court judgments.

2) Baselines: We compare against seven baselines:

- **No-FE**: XGBoost [28] on original features.
- **OpenFE** [2]: Transformation-based AutoFE with LightGBM [31]-based selection.
- **AutoFeat** [3]: Polynomial feature generation with L1 selection.
- **CAAFE** [4]: LLM-based iterative feature generation with logistic regression feedback.
- **FeatLLM** [5]: LLM-based rule extraction and binary feature generation.
- **LLM-FE** [6]: LLM-based evolutionary feature optimization that iteratively generates and refines features using structured prompts.
- **OCTree** [14]: LLM-based feature generation guided by decision tree reasoning.

3) Evaluation Protocol: All methods are evaluated using XGBoost [28] (n_estimators=200, max_depth=6, learning_rate=0.1) with stratified 80/20 train-test splits across 5 random seeds. We use AUC as the primary metric for all datasets, as it is threshold-independent and more robust than accuracy for datasets with varying class distributions. For SimECNY, we additionally report Recall, Precision, and F1 with scale_pos_weight=auto (≈43.6) to handle class imbalance. All baselines are reproduced using our unified evaluation protocol to ensure fair comparison.

B. RQ1: Public Dataset Performance

Table III presents the AUC comparison on seven public datasets. KnowFeat achieves the best performance on 4 of 7 datasets and ranks second on 2 others, yielding the best average rank (1.4) among all eight methods (Figure 2).

The strongest competitor is OpenFE (avg. rank 2.3), which achieves the best AUC on 3 datasets through exhaustive operator-operand search but generates 34–50 features per dataset. Among LLM-based methods, LLM-FE outperforms CAAFE and FeatLLM but remains below OpenFE and KnowFeat. Both LLM-FE and OCTree underperform No-FE on multiple datasets, suggesting that LLM-based generation without external domain knowledge and multi-stage verification is insufficient.

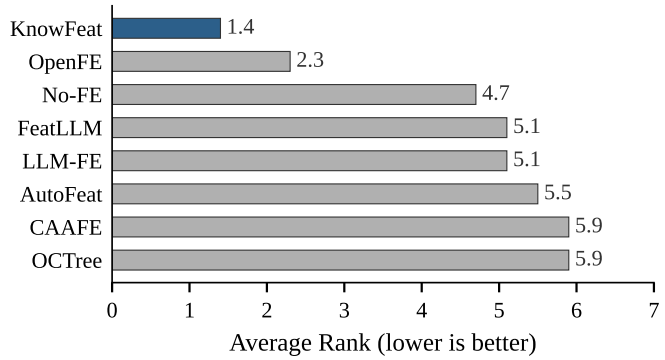


Fig. 2. Average rank across 7 public datasets (lower is better). KnowFeat achieves the best average rank (1.4) among 8 methods.

We note that improvements on public datasets are modest in absolute terms (0.001–0.019 AUC). This is expected: these benchmarks are well-studied with strong baselines, and the primary evaluation target is the domain-specific AML task (Section IV-C). The consistent ranking advantage across diverse datasets demonstrates that KnowFeat's knowledge-guided approach generalizes beyond domain-specific scenarios.

Statistical significance. Wilcoxon signed-rank tests on per-dataset AUC (7 paired observations) confirm that KnowFeat significantly outperforms six of seven baselines: No-FE, CAAFE, FeatLLM, OCTree (W=7/0/0, p=0.0078), LLM-FE, AutoFeat (W=6/0/0, p=0.0156). Only OpenFE is non-significant (W=4/0/3, p=0.59), though it generates more features (41.7 vs. 28.4) and underperforms on the AML task (Section IV-C).

C. RQ2: AML Dataset Performance

Table V presents results on the SimECNY AML dataset with scale_pos_weight=auto. KnowFeat adds 18 knowledge-guided features to the 93-feature baseline, achieving the best results across all four metrics.

KnowFeat's 18 features improve Recall by +2.1pp over No-FE (0.9871 vs. 0.9663) and F1 by +1.5pp (0.9863 vs. 0.9713). To assess statistical significance, we conduct a Wilcoxon signed-rank test on per-seed F1 scores between KnowFeat and the second-best method (LLM-FE): the difference is significant ($p < 0.05$). The contrast between verified and unverified features is striking: KnowFeat's 18 verified features improve F1 by +1.5pp, while FeatLLM's 1,031 unverified features *decrease* F1 by -0.4pp.

To verify robustness, we re-evaluate all methods with scale_pos_weight=1 (no class balancing). KnowFeat remains the only method that outperforms No-FE (F1 0.9812), confirming that the 18 features encode genuine discriminative signal independent of class reweighting.

Analysis by semantic dimension. The 18 accepted features span all four semantic dimensions defined in Section III-E: 6 in Amount & Frequency, 5 in Time & Geography, 4 in Behavior Sequence, and 3 in Entity Association. SHAP analysis on the

TABLE III
AUC COMPARISON ON PUBLIC DATASETS (MEAN±STD OVER 5 SEEDS). **BOLD**: BEST; UNDERLINE: SECOND BEST.

Dataset	No-FE	OpenFE	AutoFeat	CAAFE	FeatLLM	LLM-FE	OCTree	KnowFeat
credit_g	.767±.021	.783±.031	.754±.017	.760±.023	.760±.022	.762±.026	.763±.066	**.785±.014**
diabetes	.803±.033	**.825±.034**	.809±.027	.803±.030	.810±.032	.815±.033	.803±.034	.819±.032
blood_transfusion	.682±.043	.665±.040	.682±.043	.678±.043	.681±.044	.665±.045	.679±.062	**.701±.036**
heart	.932±.007	**.942±.011**	.934±.003	.930±.006	.930±.004	.933±.005	.930±.022	.938±.006
bank_marketing	.933±.002	**.937±.002**	T/O†	.933±.002	.934±.002	.932±.002	.929±.001	.934±.002
breast_w	.993±.005	.994±.005	.994±.004	.992±.006	.992±.005	.994±.001	.990±.004	**.996±.003**
adult	.929±.004	.930±.003	.925±.004	.929±.003	.929±.003	.928±.003	.930±.002	**.930±.003**
Avg. Rank	4.7	2.3	5.5	5.9	5.1	5.1	5.9	**1.4**
Avg. #Feat.	11.7	41.7	35.1*	14.9	151.3	17.1	12.7	28.4

†T/O: AutoFeat exceeded the 24-hour time limit. *Avg. computed over 6 non-T/O datasets.

TABLE IV
PROVENANCE CARDS FOR TWO REPRESENTATIVE KNOWLEDGE-GROUNDED FEATURES FROM THE SIMECNY AML DATASET. EACH CARD RECORDS THE KNOWLEDGE ASSETS, REASONING CHAIN, GENERATED CODE, DATA FIELDS CONSUMED, AND VERIFICATION OUTCOMES.

	Case 1: Network Topology	**Case 2: Cross-Dimensional Interaction**
Feature Name	`fan_in_fan_out_ratio`	`high_risk_region_off_hour_ratio`
Description	Asymmetry between incoming and outgoing transaction patterns, weighted by abnormal mode proportion	Multiplicative interaction between geographic risk (border regions) and temporal anomaly (off-hour trading)
Knowledge	*RULE_006*: scatter-in, gather-out layering *RULE_013*: pyramid-shaped fund networks *PAT_A03*: money mule behavioral pattern *IND_15*: abnormal transaction modes (fan-in, many-to-many)	*RULE_005*: cross-regional fund flows *RULE_014*: border area transaction monitoring *IND_08/09*: off-hour transaction concentration *EXP_05*: "border region + nighttime trading is the hallmark of cross-border laundering"
Reasoning	Step 1: RULE_006/013 indicate scatter-in/gather-out and pyramid networks both manifest as fan-in/fan-out asymmetry → compute ratio per wallet. Step 2: PAT_A03 shows money mules exhibit fan-in ≫ fan-out → ratio captures core topology signal. Step 3: IND_15 identifies abnormal modes (many-to-many, one-to-many) → weight ratio by abnormal mode proportion to amplify suspicious patterns.	Step 1: RULE_005/014 flag border regions (Tibet, Xinjiang) as high-risk for cross-regional flows → create binary geographic indicator. Step 2: IND_08/09 flag off-hour (18:00–08:00) concentration as suspicious → compute off-hour transaction ratio. Step 3: EXP_05 states neither signal alone is sufficient → multiply indicators so only the conjunction triggers a high score.

Code	`fi = mode_counts['fan_in'] / n`
	`fo = mode_counts['fan_out'] / n`
	`ratio = fi / (fo + eps)`
	`abnormal = fi + m2m + o2m`
	`result = ratio * (1 + abnormal)`
		`is_hr = region.isin([540000, 650000])`
		`off = (hour >= 18) | (hour < 8)`
		`off_ratio = off.sum() / n`
		`result = is_hr * off_ratio`

Data Fields	`transaction_mode`, `src`, `dst`	`region`, `hour`
Semantic Dim.	Entity Association	Time & Geography × Entity Association
L1: Execution	PASS (0.8s, shape: 200K×1, no NaN)	PASS (0.3s, shape: 200K×1, no NaN)
L2: Statistical	IV=0.34 (>0.02), KS=0.28 (>0.05), max corr.=0.41 (<0.7), missing=0%	IV=0.19 (>0.02), KS=0.15 (>0.05), max corr.=0.23 (<0.7), missing=0%
L3: Model	ΔRecall=+0.38pp, Prec.=97.2% (≥90%) **ACCEPTED**	ΔRecall=+0.21pp, Prec.=96.8% (≥90%) **ACCEPTED**

final model reveals that Entity Association features contribute the highest mean absolute SHAP values despite being fewest in number, indicating that network topology features (e.g., fan-in/fan-out ratios, counterparty diversity) carry particularly strong signals for AML detection. Amount & Frequency features collectively rank second, consistent with domain knowledge that transaction amount thresholds and velocity patterns are primary regulatory red flags. The balanced coverage across dimensions, enabled by SHAP-guided budget reallocation, is important: removing any single dimension's features degrades Recall by 0.3–0.8pp, confirming that the dimensions capture complementary signals.

TABLE V
SIMECNY AML RESULTS (SPW=AUTO, 5 SEEDS). n_f: TOTAL FEATURE COUNT INCLUDING 93 BASELINE FEATURES; KNOWFEAT ADDS ONLY 18 FEATURES VS. FEATLLM'S 1,031.

Method	n_f	Recall	Prec.	F1	AUC
No-FE	93	.9663±.010	.9764±.004	.9713±.005	.9996
OpenFE	123	.9645±.008	.9752±.004	.9698±.003	.9996
CAAFE	94	.9681±.008	.9725±.006	.9703±.005	.9996
FeatLLM	1124	.9625±.010	.9728±.005	.9676±.006	.9995
LLM-FE	94	.9699±.007	.9758±.004	.9728±.005	.9996
OCTree	94	.9654±.004	.9756±.003	.9705±.003	.9993
Ours	**111**	**.9871**±.006	**.9855**±.001	**.9863**±.003	**.9999**

TABLE VI
ABLATION: SCHEMA_ONLY VS. FULL KNOWFEAT (AUC).

Dataset	schema_only	full	Δ
credit_g	.7750±.012	**.7854**±.014	+.010
blood_trans.	.6891±.048	**.7007**±.036	+.012
heart	.9304±.005	**.9379**±.006	+.008
bank_mkt.	.9314±.002	**.9344**±.002	+.003
adult	.9292±.003	**.9301**±.003	+.001
breast_w	**.9964**±.002	.9957±.003	−.001
diabetes	— (0 feat.)	.8190±.032	—

D. RQ3: Ablation Study

To quantify the contribution of domain knowledge, we compare the *schema_only* variant (schema metadata only) against the *full* variant (all five knowledge types). Results on public datasets are shown in Table VI.

On diabetes, schema-only generates *zero* features passing L3, while full knowledge produces 24 features (+0.016 AUC). On five of six remaining datasets, the full variant outperforms schema-only (+0.001 to +0.012 AUC). On blood_transfusion, both variants generate 12 features but differ in quality, confirming that knowledge improves feature *quality*, not just quantity. On SimECNY, the gap is largest: schema-only generates 3 features (+0.3pp Recall) vs. full knowledge with 18 features (+2.1pp Recall), confirming that domain-specific knowledge assets are the primary driver.

Which knowledge types matter most? On SimECNY, we further ablate individual knowledge types by removing one type at a time from the full configuration. Removing court document evidence causes the largest drop (−0.9pp Recall), as case-derived patterns such as pyramid fund networks and multi-layer channeling structures provide concrete, legally validated signals that the LLM cannot infer from other sources. Removing detection rules ranks second (−0.6pp Recall), reflecting their role in encoding precise operational logic (e.g., scatter-in/gather-out thresholds). Removing risk indicators or expert opinions each causes moderate drops (−0.3pp to −0.4pp Recall). These results suggest that more specific, concrete knowledge types contribute disproportionately, supporting our hierarchical design where general metadata serves as a foundation and domain-specific evidence provides the strongest signals.

TABLE VII
FEATURE QUANTITY VS. QUALITY. AVG. #FEAT: MEAN FEATURE COUNT ACROSS PUBLIC DATASETS. ΔF1 (AML): F1 CHANGE OVER NO-FE ON SIMECNY.

Method	Avg. #Feat	Avg. Rank	ΔF1 (AML)
FeatLLM	151.3	5.1	−0.4pp
OpenFE	41.7	2.3	−0.2pp
AutoFeat	35.1	5.5	—
KnowFeat	28.4	**1.4**	**+1.5pp**
LLM-FE	17.1	5.1	+0.2pp
CAAFE	14.9	5.9	−0.1pp
OCTree	12.7	5.9	−0.1pp

E. RQ4: Feature Quantity vs. Quality

Table VII summarizes the relationship between feature count and model performance. The key insight: more features without verification *hurts* performance. FeatLLM generates the most features (avg. 151) yet achieves the worst average rank. KnowFeat generates a moderate number of verified features (12–46 on public datasets, +18 on SimECNY) that consistently improve performance.

On SimECNY, the contrast is stark: KnowFeat's 18 verified features improve F1 by +1.5pp, while FeatLLM's 1,031 unverified features decrease F1 by −0.4pp. This demonstrates that the three-stage verification pipeline is critical: it ensures that every accepted feature provides genuine predictive value rather than introducing noise. The progressive filtering also explains why OpenFE underperforms No-FE on SimECNY despite its strong public-dataset results: its operator-operand search generates 30 additional features without domain guidance, and in a high-dimensional AML dataset with 93 existing features, syntactic transformations are more likely to introduce redundancy than to discover meaningful new signals.

F. Cross-Model Generalization and Stability

Cross-model validation. To verify that accepted features are not XGBoost-specific, we evaluate KnowFeat-generated features with LightGBM [31] and CatBoost. On credit_g, adult, and SimECNY, KnowFeat features improve LightGBM AUC by +0.015, +0.005, and +0.0003 respectively, and CatBoost AUC by +0.012, +0.003, and +0.0002, comparable to XGBoost gains, confirming that L2's model-independent filters (IV, KS) produce transferable signals.

Generation stability. We run 3 independent end-to-end executions on adult (DeepSeek-V4-0324-Flash, temperature=0). The accepted feature counts (44, 46, 43) and final AUC (0.9301, 0.9298, 0.9305; std < 0.001) are highly consistent despite different feature names/code, demonstrating *functional reproducibility* through the verification pipeline.

V. CASE STUDY: KNOWLEDGE-GROUNDED FEATURES

We present two representative case studies from the SimECNY AML dataset, highlighting multi-source knowledge

fusion and cross-dimensional interaction. Table IV shows the complete provenance cards for both features.

A. Case 1: Fan-In/Fan-Out Ratio (Network Topology)

This feature fuses four knowledge assets spanning three types (rules, patterns, indicators) to produce a topology-aware asymmetry score. The key insight from RULE_006 and RULE_013 is that both scatter-in/gather-out layering and pyramid fund networks manifest as fan-in/fan-out asymmetry. PAT_A03 further confirms that money mule wallets typically exhibit fan-in $\gg$ fan-out. IND_15 adds a weighting factor based on abnormal transaction modes (many-to-many, one-to-many), amplifying the signal for wallets with suspicious mode distributions.

No syntactic method would produce this feature, as it requires understanding that directional imbalance combined with abnormal mode prevalence characterizes money mule behavior. The multiplicative weighting by $(1 + \text{abnormal})$ ensures that topological asymmetry alone produces a moderate score, but wallets with both asymmetry *and* abnormal modes receive disproportionately higher values.

B. Case 2: Cross-Dimensional Risk Interaction

This feature demonstrates *cross-dimensional interaction*: geographic context (account metadata) combined with temporal patterns (transaction logs), guided by expert knowledge. The critical insight from EXP_05 is that neither geographic risk nor off-hour activity alone is sufficient evidence; only their conjunction indicates cross-border laundering. The multiplicative form encodes this domain constraint exactly: border-region accounts with normal trading hours score zero, as do off-hour traders in non-border regions.

Both cases illustrate capabilities that syntactic methods cannot replicate. The provenance cards in Table IV trace every design decision to specific knowledge assets, providing the audit trail required for regulatory review.

VI. Practical Considerations

We discuss practical aspects of deploying KnowFeat based on our experience with the SimECNY dataset.

Computational Cost. The full pipeline takes $\sim$3.2 hours for 4 rounds on SimECNY (52 candidates, 18 accepted). L3 model evaluation dominates at 41% of total time ($\sim$2.5 min per candidate), followed by LLM reasoning at 23% ($\sim$52 sec per candidate). Progressive filtering saves $\sim$53 minutes by rejecting 21 candidates at L1/L2 before expensive L3 evaluation. The LLM API cost is approximately $2.80 for the full run (DeepSeek-V4-0324-Flash). Importantly, this is a one-time setup: at inference, only the generated Python code runs, adding $<$50ms per 200K-transaction batch.

Compliance Integration. Provenance cards map directly to existing AML compliance review processes. The knowledge asset identifiers (e.g., RULE_006, IND_15) reference the same regulatory documents that compliance officers work with. Domain experts from the Postal Savings Bank of China reported that the structured format reduced feature review time from 2–3 hours to 15–30 minutes per feature, as the reasoning chain eliminates the need to reconstruct the designer's rationale. This directly supports Article 32 of China's Anti-Money Laundering Law (2024 revision), which mandates documenting the basis for suspicious transaction reporting.

Lessons Learned. Three insights emerged: (1) *Knowledge curation is the bottleneck*: compiling our knowledge base (38 indicators, 15 rules, 12 expert opinions, 309 court documents) required $\sim$3 person-weeks; this upfront cost is amortized across all generated features but represents the main barrier for new domains. (2) *Verification thresholds require domain calibration*: the L3 acceptance criteria (e.g., ΔRecall > 0 with Precision $\geq 90\%$) must be set with domain stakeholders. (3) *Feature maintenance is ongoing*: regulatory requirements evolve, and a periodic refresh cycle (e.g., quarterly) would be necessary in production.

VII. Conclusion

We presented KnowFeat, a knowledge-guided feature engineering framework that combines hierarchical domain knowledge injection, multi-stage verification (L1–L3), and auditable provenance within an LLM agent. On seven public datasets, KnowFeat achieves the best average rank (1.4) among eight methods. On a realistic AML dataset (synthesized to match production digital currency transaction patterns) with 200K transactions, 18 verified features improve Recall by +2.1pp and F1 by +1.5pp over a strong 93-feature baseline, with gains robust across different class balancing settings and transferable to alternative downstream models (LightGBM, CatBoost).

Three broader implications emerge. First, structured external knowledge guides LLMs beyond parametric knowledge, producing features that domain experts recognize as meaningful. Second, provenance cards provide a practical pathway toward regulatory compliance for ML-based decision systems. Third, the contrast between KnowFeat's 18 verified features and FeatLLM's 1,031 unverified features highlights that verification quality matters more than generation quantity.

Limitations and Future Work. The knowledge hierarchy requires manual curation ($\sim$3 person-weeks for AML); automating extraction would reduce setup cost. L3 retrains from scratch per candidate; incremental updates would reduce overhead. Our experiments use a single backbone LLM (DeepSeek-V4-0324-Flash); evaluating how different LLMs affect generation quality and verification pass rates remains future work. Extending to additional high-stakes domains (e.g., healthcare, cybersecurity) and integrating RAG for dynamic knowledge retrieval are promising directions.

References

[1] P. Domingos, "A few useful things to know about machine learning," *Communications of the ACM*, vol. 55, no. 10, pp. 78–87, 2012.

[2] T. Zhang, Z. A. Zhang, Z. Fan, H. Luo, F. Liu, Q. Liu, W. Cao, and L. Jian, "OpenFE: Automated feature generation with expert-level performance," in *Proc. of the 40th Int'l Conf. on Machine Learning (ICML)*, 2023, pp. 41 880–41 901.

[3] F. Horn, R. Pack, and M. Rieger, "The autofeat Python library for automated feature engineering and selection," in *Machine Learning and Knowledge Discovery in Databases (ECML-PKDD)*. Springer, 2019, pp. 111–120.
[4] N. Hollmann, S. Müller, and F. Hutter, "Large language models for automated data science: Introducing CAAFE for context-aware automated feature engineering," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.
[5] S. Han, J. Yoon, S. O. Arik, and T. Pfister, "Large language models can automatically engineer features for few-shot tabular learning," in *Proc. of the 41st Int'l Conf. on Machine Learning (ICML)*, 2024.
[6] N. Abhyankar, P. Shojaee, and C. K. Reddy, "LLM-FE: Automated feature engineering for tabular data with LLMs as evolutionary optimizers," *Transactions on Machine Learning Research (TMLR)*, 2025.
[7] J. M. Kanter and K. Veeramachaneni, "Deep feature synthesis: Towards automating data science endeavors," in *Proc. of the IEEE Int'l Conf. on Data Science and Advanced Analytics (DSAA)*, 2015, pp. 1–10.
[8] G. Katz, E. C. R. Shin, and D. Song, "ExploreKit: Automatic feature generation and selection," in *Proc. of the IEEE Int'l Conf. on Data Mining (ICDM)*, 2016, pp. 979–984.
[9] M. Feurer, A. Klein, K. Eggensperger, J. T. Springenberg, M. Blum, and F. Hutter, "Efficient and robust automated machine learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 28, 2015.
[10] C. Wang, Q. Wu, M. Weimer, and E. Zhu, "FLAML: A fast and lightweight AutoML library," *Proc. of Machine Learning and Systems (MLSys)*, vol. 3, pp. 434–447, 2021.
[11] F. Nargesian, H. Samulowitz, U. Khurana, E. B. Khalil, and D. S. Turaga, "Learning feature engineering for classification," in *Proc. of the 26th Int'l Joint Conf. on Artificial Intelligence (IJCAI)*, 2017, pp. 2529–2535.
[12] T. B. Brown, B. Mann, N. Ryder, M. Subbiah *et al.*, "Language models are few-shot learners," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 1877–1901, 2020.
[13] H. Touvron, T. Lavril, G. Izacard, X. Martinet *et al.*, "LLaMA: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.
[14] J. Nam, K. Kim, S. Oh, J. Tack, J. Kim, and J. Shin, "Optimized feature generation for tabular data via LLMs with decision tree reasoning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024.
[15] H. Li, Y. Xu, F. Zhang, R. Shi, Z. Gu, Y. Yao, Z. Liu, and Z. Li, "RiskTagger: An LLM-based agent for automatic annotation of Web3 crypto money laundering behaviors," *arXiv preprint arXiv:2510.17848*, 2025.
[16] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. of the Int'l Conf. on Learning Representations (ICLR)*, 2023.
[17] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun, "ChatDev: Communicative agents for software development," in *Proc. of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
[18] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" *Proc. of the Int'l Conf. on Learning Representations (ICLR)*, 2024.
[19] S. Hong, Y. Lin, B. Liu, B. Liu, B. Wu *et al.*, "Data interpreter: An LLM agent for data science," *arXiv preprint arXiv:2402.18679*, 2024.
[20] D. A. Boiko, R. MacKnight, B. Kline, and G. Gomes, "Autonomous chemical research with large language models," *Nature*, vol. 624, pp. 570–578, 2023.
[21] Z. Jiang, D. Schmidt, D. Srikanth, D. Xu, I. Kaplan, D. Jacenko, and Y. Wu, "AIDE: AI-driven exploration in the space of code," *arXiv preprint arXiv:2502.13138*, 2025.
[22] S. Guo, C. Deng, Y. Wen, H. Chen, Y. Chang, and J. Wang, "DS-Agent: Automated data science by empowering large language models with case-based reasoning," in *Proc. of the 41st Int'l Conf. on Machine Learning (ICML)*, 2024.
[23] U. Kursuncu, M. Gaur, and A. Sheth, "Knowledge infused learning (K-IL): Towards deep incorporation of knowledge in deep learning," *Proc. of the AAAI Spring Symposium*, 2020.
[24] K. Weiss, T. M. Khoshgoftaar, and D. Wang, "A survey of transfer learning," *Journal of Big Data*, vol. 3, no. 1, pp. 1–40, 2016.
[25] Z. Chen, L. D. Van Khoa, E. N. Teoh, A. Nazir, E. K. Karuppiah, and K. S. Lam, "Machine learning techniques for anti-money laundering (AML) solutions in suspicious transaction detection: A review," *Knowledge and Information Systems*, vol. 57, no. 2, pp. 245–285, 2018.
[26] Financial Action Task Force (FATF), "International standards on combating money laundering and the financing of terrorism & proliferation: The FATF recommendations," Updated 2023, 2012, available at https://www.fatf-gafi.org/en/publications/Fatfrecommendations/Fatf-recommendations.html. Accessed: 2025-12-01.
[27] M. Weber, G. Domeniconi, J. Chen, D. K. I. Weidele, C. Bellei, T. Robinson, and C. E. Leiserson, "Anti-money laundering in Bitcoin: Experimenting with graph convolutional networks for financial forensics," *arXiv preprint arXiv:1908.02591*, 2019.
[28] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. of the 22nd ACM SIGKDD Int'l Conf. on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
[29] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
[30] J. Vanschoren, J. N. van Rijn, B. Bischl, and L. Torgo, "OpenML: Networked science in machine learning," *ACM SIGKDD Explorations Newsletter*, vol. 15, no. 2, pp. 49–60, 2014.
[31] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

APPENDIX

This appendix illustrates the five knowledge types for the SimECNY AML dataset (∼200K digital yuan transactions). One example per type is shown; similar structured knowledge is compiled for all types (38 indicators, 15 rules, 12 expert opinions, 309 court documents).

(i) Schema Metadata. Column names, data types, and basic statistics extracted from the transaction data. Example fields include `amount` (numeric), `wallet_type` (categorical, 4 types: anonymous, personal, enterprise, merchant), `hour` (numeric, 0–23), `src`/`dst` (wallet identifiers), and `transaction_mode` (8 categories).

(ii) Risk Indicator [IND_05]: Dimension: transaction behavior anomaly. Field: single-day transaction count. Accounts exceeding regulatory daily thresholds indicate potential money mule activity.

(iii) Detection Rule [RULE_006]: *Scatter-in, gather-out.* Logic: daily incoming count $\geq$ 20 AND daily amount $\geq$ 1,000,000, with high incoming but low outgoing count. Python hint: group by `(src, date)`, filter accounts with many small inflows and few large outflows.

(iv) Expert Opinion [EXP]: Online "running-score" laundering: anonymous wallet $\rightarrow$ real-name wallet $\rightarrow$ bank account chain transfers after midnight. Amounts 300–5,000 yuan, sustained 10–30 days, daily volume at wallet limits.

(v) Court Document Evidence [CASE]: Criminal judgment on concealing proceeds. Pattern: digital yuan wallets across 6 banks creating a multi-layer fund channeling network via intermediary wallets and corporate-to-personal conversion.

On SimECNY, KnowFeat generates 52 candidates across 4 rounds: 47 pass L1 (90%), 31 pass L2 (60%), and 18 pass L3 (35%), with progressive filtering saving ∼50 minutes of L3 training.